SIMULATING AN AGENT-BASED GAME USING LINKED LISTS

Abstract:

In this project, I used linked lists in Java to simulate a dynamic agent-based system, where agents with distinct social behaviors (SocialAgent and AntiSocialAgent) interact within a grid-like environment. Instead of static arrays, linked lists allowed me to efficiently manage and update agents as they moved, interacted, or avoided one another. Each agent's state was stored as a node in the linked list, enabling quick access, addition, and removal of agents as they changed positions or behaviors over time. This approach not only provided a flexible structure for modeling complex interactions but also demonstrated the application of linked lists in simulating dynamic systems where entities evolve based on predefined rules. One of the key takeaways was the importance of efficient data structures in handling dynamic simulations, where performance and adaptability are crucial. Additionally, the project deepened my understanding of how linked lists can be used to model real-world scenarios, such as social dynamics, while offering greater flexibility compared to traditional data structures like arrays.

Results:

In my experiments, I created a new class called the AgentSimulation that enabled me to visualize the movement pattern and behaviour of my agents - Social and the AntiSocial agents- in a real time simulation . My LifeSimulation code was modelled to update the states of the agent (according to the rules of the simulation - a social agent moves when there are less than 4 agents within its radius ; an antisocial agent moves when there's more than 1 agent within its radius) after 50ms. Each iteration was set to run for 50 ms each time for an agent(s) to move =. I also used a command line argument to control the size of my landscape (the board for the agents to be displayed or interact on) and the numbers of agents on the board. For my random initial conditions, I tested my AgentSimulation program for a landscape size of 500 x 500 and 50 agents. I also wanted to investigate the number of loop iterations it will take for all agents to move in the last step; hence, I set a maximum iteration loop count of 5000 loops. I adjusted my code to return the number of times my loop ran to complete the simulation.

Check the video here: InitialSimulation50.mov :

For this result, my loop ran in **230** iterations.

I got my initial board state :

Exploration 1

In this experiment, I explored the impact of varying the number of agents on the number of iterations it takes for the simulation to reach a stable state. The simulation was run with 50, 100, 150, 200, and 250 agents, and the number of iterations it took for the agents to stop moving was recorded. I hypothesized that as the number of agents increases, the system would take longer to stabilize, with higher agent densities leading to more complex interactions. For each test case, I modified the AgentSimulation program to accept command-line arguments for the width and height of the landscape and the number of agents. I set it to run the simulation for a maximum of 5000 iterations. The results were collected and analyzed to understand the relationship between agent density and simulation stability.

The results confirmed the hypothesis that more agents require more iterations to stabilize. As the number of agents increased from 50 to 250, the simulation took progressively longer to stop, increasing from 147 to 1499, with the 250-agent case often approaching the maximum iteration limit of 5000. This trend highlights how agent density influences the time it takes for the system to reach a stable state, with larger agent populations introducing more complexity and longer stabilization times. The experiment demonstrated the importance of agent density in agent-based modeling and provided insights into the behavior of complex systems with varying levels of interaction. N.B. The size of the landscape was set to 500 x 500 for all test cases.

For example for the first test case: I ran **java -ea AgentSimulation 500 500 50**

Number of Agents	**Number of Loops the Simulation ran for**
50	147

100	397
150	846
200	1406
250	1499

The Average Number of Iterations Taken for a Simulation on Varying Number of Agents.

Exploration2

In this experiment, I explored the impact of varying the radius of the agents on the number of iterations it takes for the simulation to stabilize. I modified the AgentSimulation program to allow the radius of each agent to be specified via command-line arguments, alongside the number of agents and the size of the landscape. By testing different radius values, I aimed to see how the interaction range of agents would influence the time it took for the simulation to reach a stable state. Specifically, I varied the radius values for agents, specifically 5, 10, 20, 40, 80 units, keeping the number of agents constant at 50 agents, and observed how the system's behavior changed as the agents' interaction range increased or decreased. The simulation ran for up to 5000 iterations to ensure enough time for stabilization.

The results showed that as the radius of the agents increased from 5 to 80, the simulation took fewer iterations to stabilize (a radius of 5 used 210 simulations, whereas a radius of 80 only used 138). This is because a larger radius allowed agents to interact with a larger number of neighbors, leading to quicker stabilization in the system. In contrast, smaller radii resulted in more localized interactions and a slower convergence to stability. The trend was clear: larger interaction areas led to faster stabilization, likely due to the increased frequency of agent interactions. This experiment demonstrated the importance of agent radius in determining the dynamics of agent-based simulations and highlighted how the physical parameters of agents can significantly affect the overall system behavior.

My command line parameters were in the format : **java -ea AgentSimulation <width> <height> <number of agents> <agent radius>**

To run my program for the first test case: I ran **java -ea AgentSimulation 500 500 50 5**

Agent Radius	Number of Loops the Simulation ran for
5	210
10	170
20	150
40	142
80	138

Extensions:

For my extension, I made several specific changes to the agent behaviors in the simulation by introducing social and antisocial agent states, differing from the original setup where all agents followed the same behavior. Initially, all agents were neutral and moved without any preferential interaction towards or away from others. In the updated version, I modified the Agent class to differentiate between social and antisocial agents. Social agents were designed to move toward other agents, while antisocial agents were programmed to avoid other agents by increasing their distance from others. These changes were implemented by adding new conditions to the update method within the respective social and antisocial class, **where social agents would adjust their movement vector to approach nearby agents, and antisocial agents would adjust their movement vector to move in the opposite direction**.

Furthermore, I modified the AgentSimulationclass to accommodate different types of agents. Instead of creating only social agents, the simulation now randomly creates both social and antisocial agents, with the number of each type specified by command-line arguments. This allowed for flexibility in running different configurations with varying numbers of social and antisocial agents. I also adjusted the iteration logic to track how the different behaviors influenced the overall number of iterations

needed for the simulation to reach a stable state. This extension added a layer of complexity to the simulation by making agent interactions dependent on their type, leading to varied movement patterns and stabilization times based on the ratio of social to antisocial agents. As a result, the simulation now provided more insightful outcomes compared to the previous uniform agent behavior, where the dynamics were relatively simple. For this simulation. I set my number of agents to be 500 and still kept the size of the landscape to be at 500 x 500.

I also wanted to create a clear difference between the original project and this extension. Hence I ran the same command in my original project.

I got a video of running the simulation by putting all the following parameters

java -ea AgentSimulation 500 500 500

I got a video of running the simulation by putting all the following parameters

Check the video for the original project here: OriginalSimulation.mov

Check the video for the extension project here: ExtensionSimulation.mov

Acknowledgements:

I consulted a lot of Youtube videos on the use of LinkedLists , implementation and related data structures. I also went to the professor's office hours and the TA hours (Chi Hoang) for a better explanation on LinkedLists and related java syntax. I also studied the code from the class notes to get a better understanding of the related concepts. I also discussed code concepts with Emmanuel Buabeng during my project testing and some implementations.